AI Lab Experiments (1–20)

Experiment 1

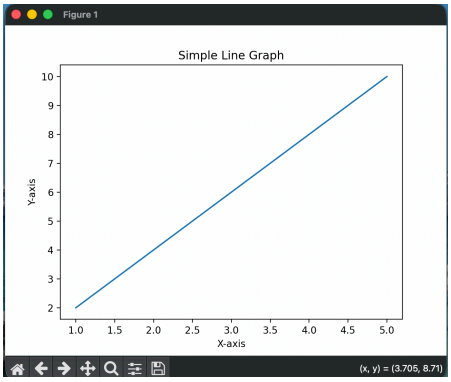
Aim:
To install the Matplotlib library and create a simple line graph for data visualization.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]

y = [2, 4, 6, 8, 10]

plt.plot(x, y)

plt.title("Simple Line Graph")

plt.xlabel("X-axis")

plt.ylabel("Y-axis")

plt.show()
```

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 2

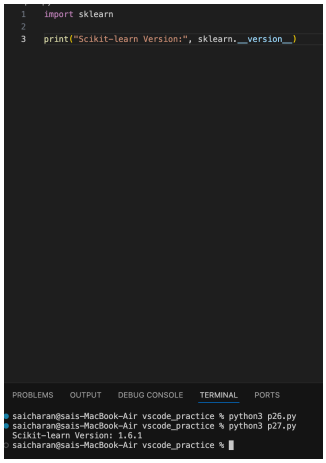
Aim:
To install the Scikit-learn library and verify its installation by displaying its version.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:
import sklearn

print("Scikit-learn Version:", sklearn.__version__)

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 3

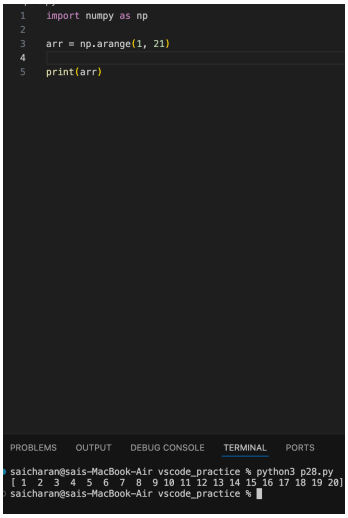
Aim:
To create and display a NumPy array containing the first 20 natural numbers.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
import numpy as np

arr = np.arange(1, 21)

print(arr)
```

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 4

Aim:
To create and display a Pandas DataFrame using student names and grades.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
import pandas as pd

data = {

   "Name": ["Sai", "Rahul", "Anu", "Priya"],

   "Grade": [90, 85, 95, 88]

}

df = pd.DataFrame(data)

print(df)
```

Output:

Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 5

Aim:
To verify the successful installation of AI libraries by displaying their installed versions.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
import numpy

import pandas

import matplotlib

import sklearn

import torch

import transformers

print("NumPy:", numpy.__version__)

print("Pandas:", pandas.__version__)

print("Matplotlib:", matplotlib.__version__)

print("Scikit-learn:", sklearn.__version__)

print("PyTorch:", torch.__version__)

print("Transformers:", transformers.__version__)
```

Output:

Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 6

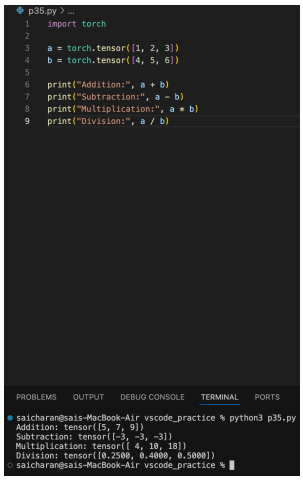
Aim:
To create tensors using PyTorch and perform basic mathematical operations on them.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
import torch

a = torch.tensor([1, 2, 3])

b = torch.tensor([4, 5, 6])

print("Addition:", a + b)

print("Subtraction:", a - b)

print("Multiplication:", a * b)

print("Division:", a / b)
```

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 7

Aim:
To load a pre-trained transformer model using Hugging Face and perform sentiment analysis on text.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

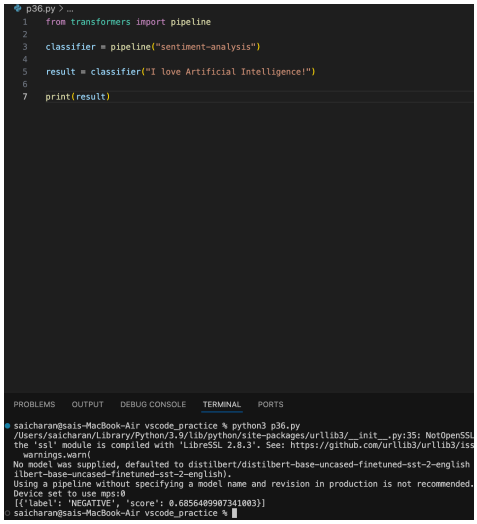
Code:
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

result = classifier("I love Artificial Intelligence!")

print(result)

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 8

Aim:
To tokenize a sentence using the BERT tokenizer**.**

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:
from transformers import BertTokenizer

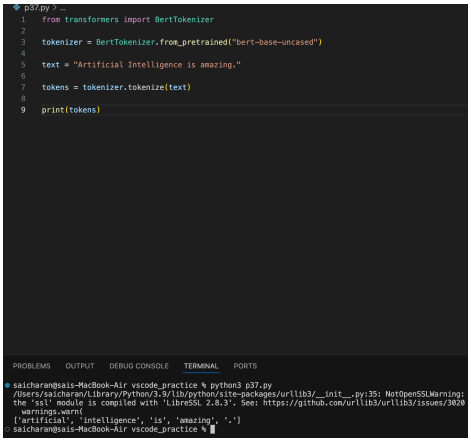
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

text = "Artificial Intelligence is amazing."

tokens = tokenizer.tokenize(text)

print(tokens)

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 9

Aim:
To generate text from a given prompt using a pre-trained GPT-2 language model.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

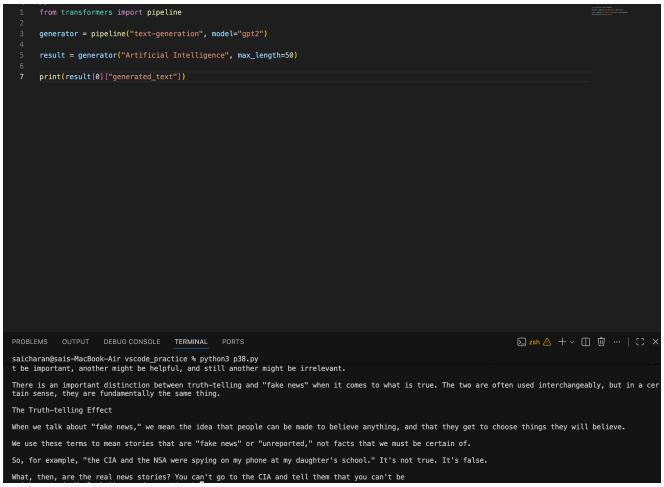
Code:
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

result = generator("Artificial Intelligence", max_length=50)

print(result[0]["generated_text"])

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 10

Aim:
To perform sentiment analysis using the Hugging Face pipeline() API with a pre-trained transformer model.

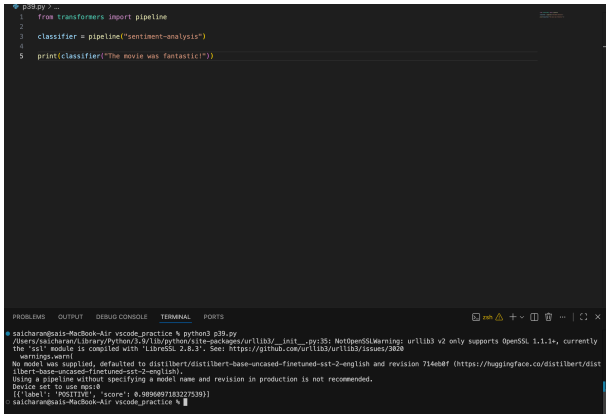
Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:
from transformers import pipeline

classifier = pipeline("sentiment-analysis")

print(classifier("The movie was fantastic!"))

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 11

Aim:
To load the BERT tokenizer and tokenize a given sentence**.**

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:
from transformers import BertTokenizer

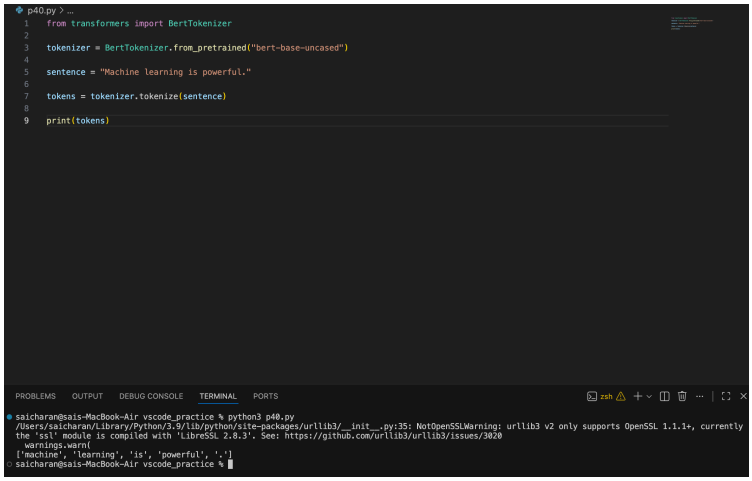
tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

sentence = "Machine learning is powerful."

tokens = tokenizer.tokenize(sentence)

print(tokens)

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 12

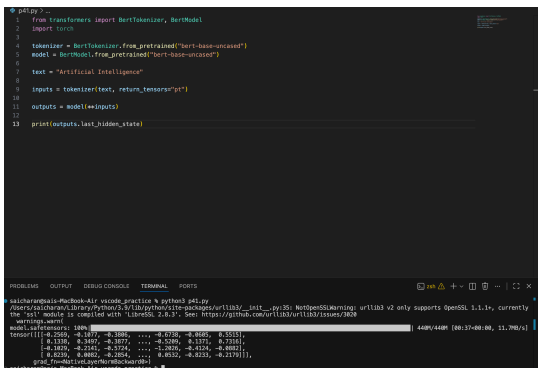
Aim:
To generate contextual word embeddings using a pre-trained BERT model.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import BertTokenizer, BertModel

import torch

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")

text = "Artificial Intelligence"

inputs = tokenizer(text, return_tensors="pt")

outputs = model(**inputs)

print(outputs.last_hidden_state)
```

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 13

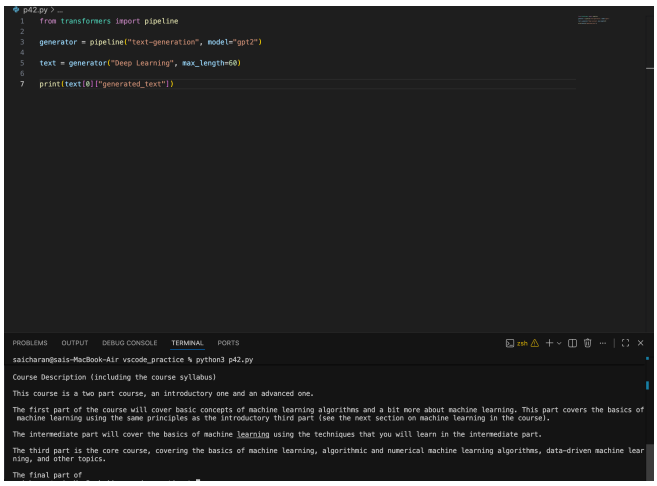
Aim:
To generate meaningful text using a pre-trained GPT-2 model.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

text = generator("Deep Learning", max_length=60)

print(text[0]["generated_text"])
```

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 14

Aim:
To perform question answering using a pre-trained transformer model**.**

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:
from transformers import pipeline

qa = pipeline("question-answering")

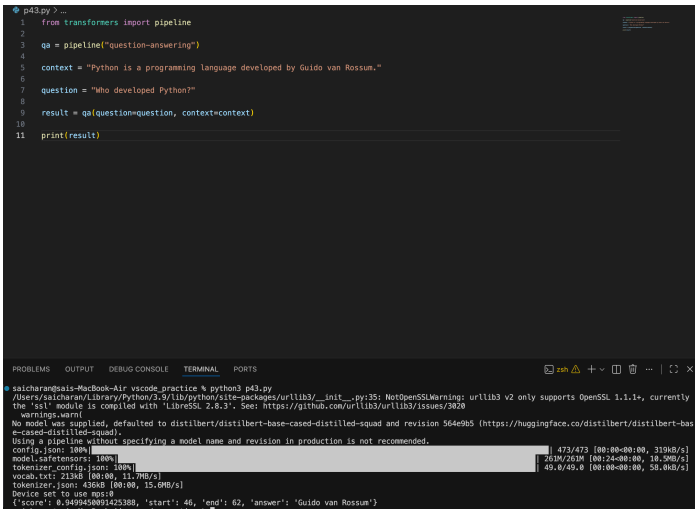
context = "Python is a programming language developed by Guido van Rossum."

question = "Who developed Python?"

result = qa(question=question, context=context)

print(result)

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 15

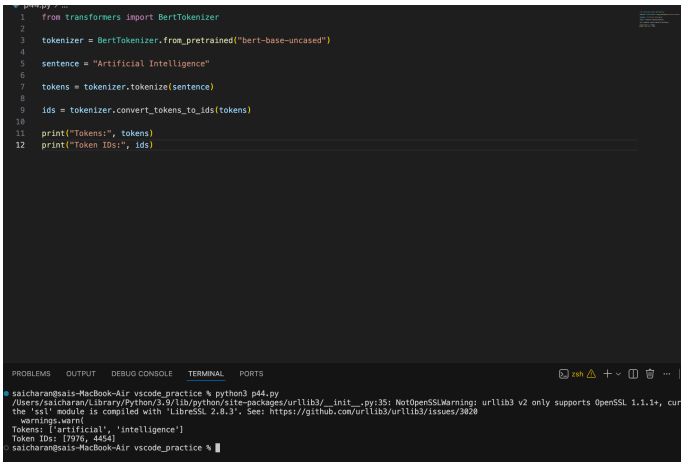
Aim:
To tokenize a sentence using the BERT tokenizer and display the generated tokens along with their token IDs.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import BertTokenizer

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

sentence = "Artificial Intelligence"

tokens = tokenizer.tokenize(sentence)

ids = tokenizer.convert_tokens_to_ids(tokens)

print("Tokens:", tokens)

print("Token IDs:", ids)
```

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 16

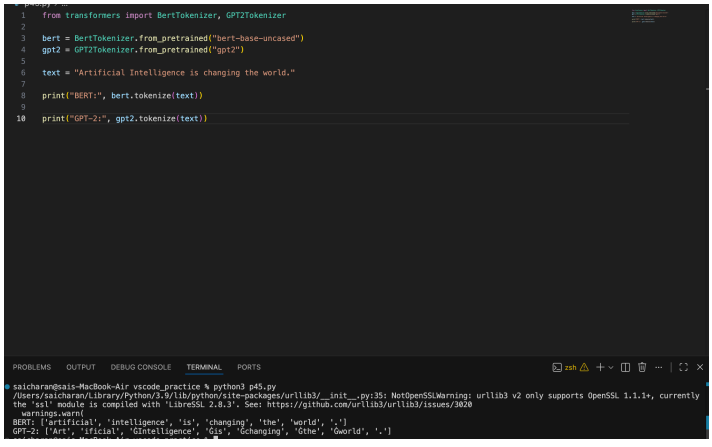
Aim:
To compare the tokenization results produced by the BERT and GPT-2 tokenizers.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import BertTokenizer, GPT2Tokenizer

bert = BertTokenizer.from_pretrained("bert-base-uncased")

gpt2 = GPT2Tokenizer.from_pretrained("gpt2")

text = "Artificial Intelligence is changing the world."

print("BERT:", bert.tokenize(text))

print("GPT-2:", gpt2.tokenize(text))
```

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 17

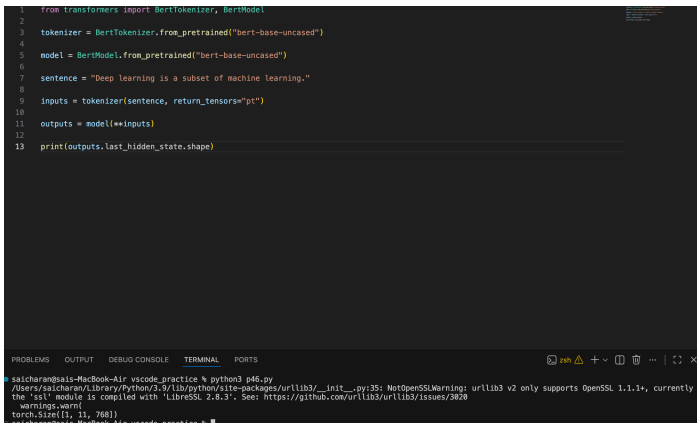
Aim:
To generate contextual embeddings for a sentence using a pre-trained BERT model.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import BertTokenizer, BertModel

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")

sentence = "Deep learning is a subset of machine learning."

inputs = tokenizer(sentence, return_tensors="pt")

outputs = model(**inputs)

print(outputs.last_hidden_state.shape)
```

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 18

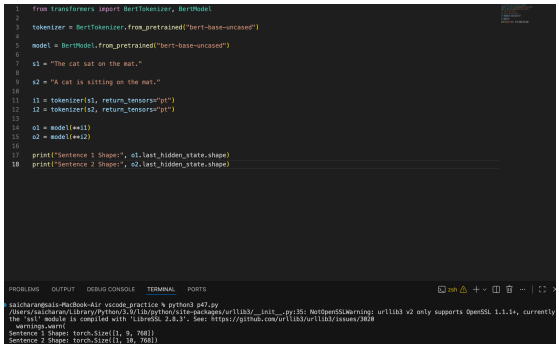
Aim:
To compare the contextual embeddings generated by BERT for two semantically similar sentences.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.

Code:

```
from transformers import BertTokenizer, BertModel

tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")

model = BertModel.from_pretrained("bert-base-uncased")

s1 = "The cat sat on the mat."

s2 = "A cat is sitting on the mat."

i1 = tokenizer(s1, return_tensors="pt")

i2 = tokenizer(s2, return_tensors="pt")

o1 = model(**i1)

o2 = model(**i2)

print("Sentence 1 Shape:", o1.last_hidden_state.shape)

print("Sentence 2 Shape:", o2.last_hidden_state.shape)
```

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 19

Aim:
To generate coherent text from a user-provided prompt using the GPT-2 language model.

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:
from transformers import pipeline

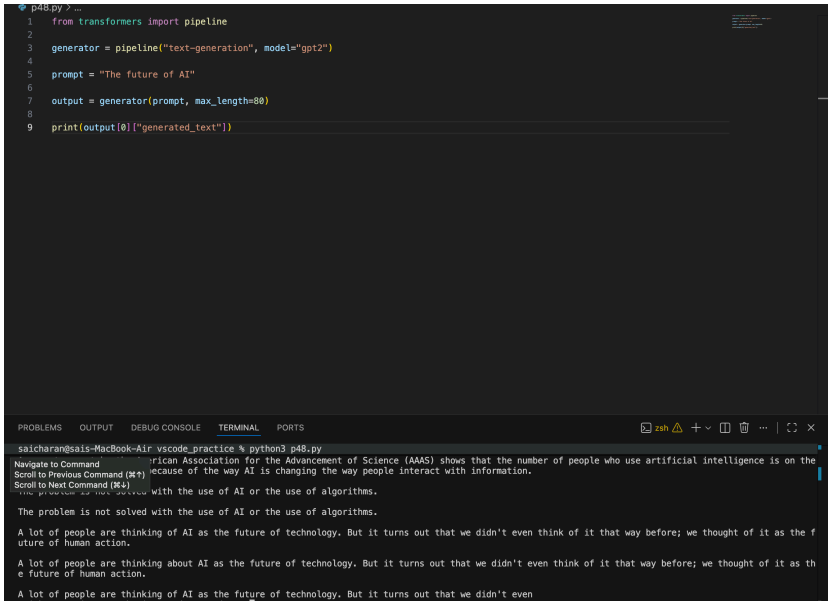
generator = pipeline("text-generation", model="gpt2")

prompt = "The future of AI"

output = generator(prompt, max_length=80)

print(output[0]["generated_text"])

Output:



Result:
The experiment was executed successfully and the expected output was obtained.

Experiment 20

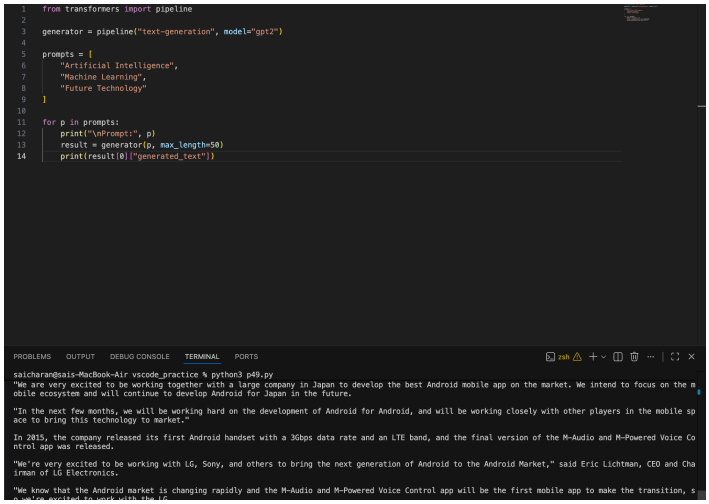
Aim:
To generate and compare responses for multiple prompts using a pre-trained Large Language Model (LLM).

Algorithm:
1. Start the program.
2. Import required libraries.
3. Execute the required steps.
4. Display the output.
5. Stop.

Code:

```
from transformers import pipeline

generator = pipeline("text-generation", model="gpt2")

prompts = [
    "Artificial Intelligence",
    "Machine Learning",
    "Future Technology"
]

for p in prompts:
    print("\nPrompt:", p)
    result = generator(p, max_length=50)
    print(result[0]["generated_text"])
```

Output:



Result:
The experiment was executed successfully and the expected output was obtained.